A PROJECT REPORT

On

Movie Recommendation System: NLP Based Recommender System

Submitted in partial fulfilment of the requirements to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA

For the award of the degree

BACHELOR of TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted by

KAITEPALLI NAGARAJU (22JK1A0543)

BATTULA VISHNU (22JK1A0527)

BIKKI VISHNU VARDHAN (22JK1A0528)

BOJJANKULA RAHUL SAI (22JK1A0529)

Under the Guidance of

K SRINIVAS REDDY

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

KITS AKSHAR INSTITUTE OF TECHNOLOGY :: YANAMADALA

(Approved by AICTE, New Delhi Affiliated to JNTUK Kakinada)

2022 – 2026

KITS AKSHAR INSTITUTE OF TECHNOLOGY

Approved by AICTE, Accredited by NAAC &Affiliated to JNTUK - Kakinada.

opp. Katuri Medical College, Yanamadala, Andhra Pradesh 522019

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

KITS AKSHAR INSTITUTE OF TECHNOLOGY

2022-2026

CERTIFICATE

This is certify that this Project work titled **"Movie Recommendation System: NLP-Based Recommender System"** is the bonafide work of KAITEPALLI NAGARAJU (22JK1A0543), BATTULA VISHNU (22JK1A0527), BIKKI VISHNU VARDHAN (22JK1A0528),RAHUL SAI BOJJANKULA(22JK1A0529) who carried out the work under the supervision and submitted in the partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering** from **JNTU Kakinada** during the academic year 2022-2026

Internal Guide

K SRINIVAS REDDY
Assistant Professor Dept .of CSE

Head of the Department

Dr. G. GURU KESAVA DAS
M. Tech, Ph. D Professor Dept .of CSE

External Guide

DECLARATION

We are here by inform that this main project entitled **"Movie Recommendation System: NLP-Based Recommender System"** has been carried out and submitted in partial fulfilment for the award to the degree of Bachelors of Technology in Computer Science & Engineering to **Jawaharlal Nehru Technical University Kakinada** under the guidance of **Mr. K SRINIVAS REDDY**, Assistant Professor Dept of Computer Science & Engineering. The work embodied in this project work is original and has been submitted in part or full for any degree of any other university.

KAITEPALLI NAGARAJU	22JK1A0543
BATTULA VISHNU	22JK1A0527
BIKKI VISHNU VARDHAN	22JK1A0528
BOJJANKULA RAHUL SAI	22JK1A0529

ACKNOWLEDGEMENT

We express our profound gratitude to Dr. GURU KESAVA DAS GOPISETTY, M.E (CSE), Ph.D. (CSE), Professor & Head of the Department of Computer Science and Engineering (CSE), for his invaluable guidance and unwavering support as our internal guide. His meticulous supervision and insightful advice have been instrumental in the successful completion of our B. Tech main project.

We are deeply indebted to Dr. G. GURU KESAVA DAS for providing us with all the necessary resources and encouragement throughout our project journey. We extend our heartfelt thanks to the college management for their continuous support and the excellent facilities provided, enabling us to carry out our project efficiently. Our sincere gratitude also goes to Dr. K. RAMA KOTAIAH, Principal, for granting us permission to undertake our main project.

Furthermore, we express our appreciation to our Director, Dr. G. SAMBASIVA RAO, our Secretary, Sri K. SEKHAR, and our Chairman, Sri K. SUBBA RAO, for their guidance and assistance during the course of our work. We are immensely thankful to the faculty and staff of the Department of Computer Science and Engineering for their valuable time, encouragement, and technical support.

Finally, we extend our deepest gratitude to our parents, friends, and well-wishers, whose unwavering support and motivation have been a driving force in our academic journey.

Institute Vision

To be a recognized premier institution in engineering education, research and application of knowledge to benefit society.

Institute Mission

- Impact technical knowledge at all levels to transform the engineers capable to address the challenges.
- Inculcate competencies for all-round development to meet industrial, technical and leadership skills.
- Provide learner-centric ambiance with collaborations.
- Promote creativity, knowledge development, ethical values and inter-personal skills.

Department of Computer Science & Engineering

Vision

To be a centre of excellence in computer engineering education, empowering Graduates as highly skilled professionals.

Mission

- **DM1:** Provide a learning Platform with emphasis on technical excellence.
- **DM2:** Organize trainings to meet industrial and emerging technologies.
- **DM3:** Install the drive for higher learning and research.
- **DM4:** Inculcate leadership and entrepreneur skills with industry collaborations.

Program Specific Outcomes (PSOs)

PSO1: Identify the data; Design suitable algorithm by using Latest software or real Time applications.

PSO2: Understanding the evolutionary changes in computing process knowledge of context aware applicability of paradigms and meet the challenges of the future.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Analyze, design, develop and test the software products and provide creative, sustainable solution.

PEO2: Use modern tools, techniques to analyze and provide solutions to the industrial problems.

PEO3: Demonstrate the managerial and leadership skills to become successful professionals.

Project Outcomes

PO-1 EngineeringKnowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO-2 Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3 Design/development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations

PO-4 Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5 Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO-6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7 Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts.

PO-8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9 Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO-10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11 Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12 Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Project Course Outcomes

C424.1: Review the literature and develop solutions for problem statement

C424.2: Implement hardware and/or software requirements for identified problems

C424.3: Design the code for the problem statement

C424.4: Test and analyze the modules of planned project.

C424.5: Write technical report and deliver presentation.

C424.6: Applyengineering and management principles to achieve project goal.

Course Outcomes – Program Outcome correlation

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2
C424.1	3	2		2		2	3						3	
C424.2	3	2		2		2			3	2	1		2	2
C424.3		3			3							2	1	
C424.4	2	2	3	2	3		2	1	3	2			3	
C424.5			3		2		2		3		2	2	3	2
C424.6						3				2		2		3

3: High 2: Medium 1: Low

CO-PO Mapping with Reasons:

C424.1 is mapped with PO1, PO2 and PO4, PO6, PO7 as basic knowledge of Engineering and problem Analysis activities are highly essential to conduct examinations on existing systems which have been using in industries as a part of and to define the problem of proposed system.

C424.2 is mapped with PO1, PO2, PO4 and PO6, PO9, PO10, PO11 as for identification, gathering analysis and classification of requirements for the proposed system, basic knowledge of engineering and Analysis steps along with complex problem analysis through the efforts of team work in order to meet the specific needs of the customer.

C424.3 is mapped with PO2, PO5 and PO12 as to conduct the literature review and to examine the relevant systems to understand and identify the merits and demerits of each too enhance and develop the proposed as per the need.

C424.4 is mapped with PO1, PO2, PO3, PO4, PO5 and PO7, PO8, PO9, PO10 because modularization and design of the project is needed after requirements elicitation. For modularization and design of the project, Basic knowledge of Engineering, Analysis capabilities, Design skills and communication is needed between team members as different modules are designed individually before integration.

C424.5 is mapped with PO3, PO5, PO7, PO9, PO11 and PO12 as to construct the project latest technologies are needed. The development of project is done

individually and in groups with well-defined communication by using the engineering and management principles.

C424.6 is mapped with PO6, PO10 and PO12 because during and after completion of the project, documentation is needed along with proper methods of presentation through understanding and application of engineering and management principles, which in turn needs well defined communication between the team members with all the ethical values.

CO-PSOs Mapping with Reasons:

C424.1 is mapped with PSO1 as examining of existing systems and identification of the problem is a part of Application Development activity and identification of evolutionary changes in latest technologies.

C424.2 is mapped with PSO1 and PSO2 as identifying and classifying the requirements is a part of Application development and evolutionary computing changes and also follows ethical principles.

C424.3 is mapped with PSO1 as review of literature is a part of application development activity by recognizing the computing technologies and their evolutionary changes.

C424.4 is mapped with PSO1 because modularization and logical design is also a part of Application development and follows computing changes using Deep learning technology.

C424.5 is mapped with PSO1, PSO2 as Testing, Development and Integration of project activities are part of Application development and follows ethical principles.

C424.6 is mapped with PSO2 as for project documentation and presentation; the project team members apply the professional and leadership qualities.

INDEX

ABSTARCT

Page no

CHAPTER 1 INTRODUCTION 01-04

1.1 Back ground and Motivation 02

1.2 Problem Statement 02-03

1.3 Objectives of the Study 03

1.4 Scope of the Project 03-04

1.5 Organisation of the report 04

CHAPTER 2 Literature Survey 05-08

2.1 Overview of Recommendation system 06

2.2 Content-based Filtering 06-07

2.3 Collaborative Filtering 07

2.4 Hybrid Approaches 07-08

2.5 NLP Techniques in recommendation 08

2.6 Summary Table 08

CHAPTER 3 System Analysis 09-11

3.1 Survey of Existing Tools 10

3.2 Identified Limitations 10

3.3 Proposed system Overview 11

3.4 Comparative Feature Analysis 11

CHAPTER 4 SYSTEM REQUIREMENTS 12-14

4.1 Functional Requirements 13

4.2 Non-Functional Requirements 13

4.3 Hardware requirements 14

4.4 software requirements 14

CHAPTER 5 System study 15-17

5.1 Feasibility study 16

5.2 Feasibility Analysis 16-17

CHAPTER 6 System Design 18-24

6.1 System Architecture 19

6.2 UML Diagrams 20

6.3 Class Diagram 21
6.4 Sequence Diagram 22
6.5 Activity Diagram 23
6.6 Colloboration Diagram 24
CHAPTER 7 Input and Output design 25-27
7.1 Input Design 26
7.2 Output Design 27
CHAPTER 8 Implementation 28-31
8.1 Development Environment 29
8.2 Module Description 29-30
8.3 NLP pipeline 31
CHAPTER 9 Software Environments 32-
9.1 Python 33-46
9.2 Source code 47-50
CHAPTER 10 Results/Discussions 51-62
10.1 System Test 52-56
10.2 Screen shorts 57-62
CHAPTER 11 Conclusion 63-65
11.1 Summary of Findings 64
11.3 Future scope 65
CHAPTER 12 References/Bibliography 66-68

LIST OF FIGURES

Figure No.	Figure name	page
Fig 1	System architecture	32
Fig 2	UML Diagrams	33
Fig 3	Class Diagram	34
Fig 4	Sequence Diagram	35
Fig 5	Activity Diagram	36
Fig 6	Colloboration Diagram	37
Fig 9.1-9.8	Python Download	33-46
Fig 10.1-10.6	Screenshorts	57-62

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSV	Comma-Separated Values
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
EDA	Exploratory Data Analysis
GMM	Gaussian Mixture Model
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
NaN	Not a Number
PCA	Principal Component Analysis
REST	Representational State Transfer
RT	Rotten Tomatoes

Movie Recommendation System: NLP Based Recommender System

ABSTRACT

The challenge of discovering movies that genuinely match a viewer's taste has grown harder as streaming platforms now carry catalogs of tens of thousands of titles. Browsing by genre is too coarse, popularity charts recycle the same well-known titles, and services that use collaborative filtering are useless to anyone without an established viewing history. This project Movie Recommendation System: NLP Based Recommender System tackles this problem by building a content-based recommendation engine that understands films through the language used to describe them.

The system processes the TMDB 5000 Movie Dataset, extracting genre tags, thematic keywords, plot overviews, top-billed cast members, and director credits for each of the approximately 4,800 films. These features are merged into a single descriptive tag string per film and then cleaned using Natural Language Processing — tokenisation, lowercase normalisation, and English stopword removal via NLTK. The cleaned corpus is vectorised using TF-IDF (Term Frequency-Inverse Document Frequency) with up to 10,000 features and 1-2 word n-grams. Cosine similarity is computed across all film pairs, producing a 4,800 x 4,800 similarity matrix serialised to disk and loaded instantly on subsequent runs.

The recommendation engine is exposed through a Flask web application with a dark-themed, responsive interface. Users type any movie title into the search bar — aided by live autocomplete — and receive the ten most similar films in under a second. Each recommendation card displays the movie poster (fetched from the OMDb API), title, and similarity score. A detail modal provides additional context: full plot, cast, director, IMDB rating, Rotten Tomatoes score, runtime, content rating, awards, and a direct YouTube trailer search link.

The project demonstrates that NLP-driven content-based filtering can deliver contextually meaningful recommendations without any user data, making it immediately useful from the very first query. The architecture is modular, the pipeline is fully reproducible, and the system is ready for deployment with minor infrastructure hardening.

Keywords: Content-based filtering, NLP, TF-IDF, cosine similarity, Flask, movie recommendation, TMDB dataset, OMDb API, Python, scikit-learn, NLTK.

CHAPTER-1
INTRODUCTION

1. INTRODUCTION

1.1 Background and Motivation

Every major streaming service today shows a personalised recommendation row 'Because you watched...' or 'Recommended for you.' Behind that deceptively simple feature sits a sophisticated recommendation engine that maps viewer preferences to a catalogue of thousands of titles in milliseconds. The scale of the problem is striking: Netflix alone carries over 17,000 titles as of 2024, and the average viewer spends nearly 20 minutes pe1.1 Background and Motivation

Every major streaming service today shows a personalised recommendation row — 'Because you watched...' or 'Recommended for you.' Behind that deceptively simple feature sits a sophisticated recommendation engine that maps viewer preferences to a catalogue of thousands of titles in milliseconds. The scale of the problem is striking: Netflix alone carries over 17,000 titles as of 2024, and the average viewer spends nearly 20 minutes per session deciding what to watch before giving up. A good recommendation system shrinks that decision window dramatically and, as a result, keeps users engaged.

Two dominant paradigms power most production recommendation systems today. Collaborative filtering looks at the aggregate behaviour of all users and concludes that two viewers who both enjoyed The Godfather and Goodfellas will likely share other tastes too. Content-based filtering, on the other hand, looks directly at the properties of the items themselves — the genre, themes, cast, director, and descriptive language — and recommends titles that share those properties. Each approach has genuine strengths and real limitations that inform the design choices made in this project.

CineMatch explores the content-based approach specifically, applying Natural Language Processing to film metadata. The motivation is practical: a system that requires no user history is immediately deployable, immediately useful, and raises no privacy concerns. It also has an appealing interpretability property — every recommendation can be explained in terms of shared textual features, not opaque user-behaviour patterns.

1.2 Problem Statement

The volume of available films makes unassisted discovery genuinely difficult. Genre browsing is too coarse-grained — the label 'thriller' encompasses everything from slow-burn psychological dramas to high-speed action chases. Popularity charts promote the same

titles endlessly and systematically neglect deep-catalogue films that share strong thematic DNA with what a viewer enjoys. Collaborative filtering, while powerful at scale, is completely blind in a cold-start scenario: a new deployment with no usage history has nothing to learn from.

The specific problem this project addresses is: given a movie title that a viewer has enjoyed, how can we automatically identify the ten most thematically and stylistically similar titles from a large corpus — without relying on any user history, ratings data, or external paid services — and present those results through a fast, visually polished web interface?

1.3 Objectives

The principal objectives of this project are as follows:

- To design and implement an NLP-driven content-based movie recommendation engine using TF-IDF vectorisation and cosine similarity on the TMDB 5000 dataset.
- To build an NLP preprocessing pipeline using NLTK that cleans, tokenises, and removes stop words from combined movie feature strings.
- To develop a Flask-based web application with live autocomplete search, responsive movie cards, a detail modal, and a trending row.
- To integrate the OMDb API for real-time retrieval of movie posters, IMDB ratings, full plot summaries, and trailer search links.
- To persist the trained similarity model using Python's pickle module, eliminating retraining overhead on subsequent application launches.
- To conduct systematic unit, integration, and end-to-end testing and document results with test cases and sample recommendation outputs.

1.4 Scope of the Project

Cine Match is scoped to content-based movie recommendation using a static dataset of approximately 4,800 films from the TMDB 5000 corpus. It operates without any user account system, login, or persistent user data. The recommendation logic is intentionally transparent and inspectable — every similarity score is derived from interpretable textual features, not from opaque neural representations.

The project does not extend to TV series, documentaries, music, or books. The web interface is a single-page application served by Flask's development server and is intended for local or intranet deployment during this academic iteration. Extending it to a production cloud deployment would require containerisation and a production-grade WSGI server, both of which are described as future work in Chapter 7.

1.5 Organisation of the Report

Chapter 2 presents a structured literature review covering the evolution of recommendation system paradigms and the NLP techniques that underpin content-based approaches. Chapter 3 analyses the limitations of existing systems and formally defines the proposed solution. Chapter 4 presents the full system design including architecture, UML diagrams, data model, and I/O design. Chapter 5 details the implementation — the development environment, module breakdown, NLP pipeline, and annotated source code walkthrough. Chapter 6 presents testing, evaluation, sample results, and screenshot descriptions. Chapter 7 concludes the project and outlines future work.

CHAPTER-2
LITERATURE
SURVEY

2. LITERATURE SURVEY

2.1 Overview of Recommendation Systems

Recommendation systems are information filtering tools designed to reduce information overload by predicting items a user is likely to find relevant. The formal study of these systems began in the mid-1990s, catalysed by the Group Lens project at the University of Minnesota which introduced collaborative filtering for Usenet news articles [1]. Since then, the field has expanded rapidly, driven partly by the commercial stakes involved: Netflix famously offered a one-million-dollar prize in 2006 for a 10% improvement on its recommendation accuracy, attracting hundreds of teams and producing a wave of seminal research.

Recommendation approaches are conventionally organised into three families: collaborative filtering, content-based filtering, and hybrid methods. Each family makes different assumptions about what data is available and what constitutes similarity between items or users. The choice of approach for any given deployment depends critically on data availability, interpretability requirements, and cold-start considerations.

2.2 Content-Based Filtering

Content-based filtering builds a profile of each item 2.1 Overview of Recommendation Systems

Recommendation systems are information filtering tools designed to reduce information overload by predicting items a user is likely to find relevant. The formal study of these systems began in the mid-1990s, catalysed by the GroupLens project at the University of Minnesota which introduced collaborative filtering for Usenet news articles [1]. Since then, the field has expanded rapidly, driven partly by the commercial stakes involved: Netflix famously offered a one-million-dollar prize in 2006 for a 10% improvement on its recommendation accuracy, attracting hundreds of teams and producing a wave of seminal research.

Recommendation approaches are conventionally organised into three families: collaborative filtering, content-based filtering, and hybrid methods. Each family makes different assumptions about what data is available and what constitutes similarity between items or users. The choice of approach for any given deployment depends critically on data availability, interpretability requirements, and cold-start considerations.

Content-based filtering builds a profile of each item from its inherent attributes and recommends items whose profiles are similar to items the user has interacted with — or, in the item-to-item variant employed by CineMatch, items that are mutually similar regardless of user behaviour. Pazzani and Billsus [2] provided one of the clearest accounts of this approach, noting that TF-IDF vectorisation of textual item descriptions followed by cosine similarity produces a strong and interpretable baseline.

Lops, de Gemmis, and Semeraro [3] conducted a comprehensive survey of the state of content-based recommendation and highlighted feature engineering as the most consequential design decision. They noted that raw TF-IDF on unprocessed text tends to over-weight high-frequency but low-discriminating words unless stopword filtering and normalisation are applied. This finding directly motivated the NLP preprocessing stage in CineMatch's recommender pipeline.

2.3 Collaborative Filtering

Collaborative filtering (CF) predicts a user's preference for an item based on the preferences of similar users. Koren, Bell, and Volinsky [4] produced the landmark paper on matrix factorisation for CF, demonstrating that latent factor models substantially outperform neighbourhood-based methods on the Netflix Prize dataset. Their work established the theoretical foundation for most production CF systems in use today.

The fundamental limitation of CF for this project is the cold-start problem: a system with no usage history cannot make any CF-based recommendations at all. Since CineMatch is designed as a standalone academic deployment with no historical interaction data, CF was evaluated and explicitly rejected as the primary approach. CF remains viable as a future enhancement once user interaction data accumulates.

2.4 Hybrid Approaches

Burke [5] provided a foundational taxonomy of hybrid recommendation strategies, cataloguing seven distinct ways content-based and collaborative components can be combined: weighted, switched, mixed, feature combination, cascade, feature augmentation, and meta-level. The survey established that hybrid systems generally outperform pure approaches but introduce additional complexity in both design and maintenance.

Netflix's production system, described by Gomez-Uribe and Hunt [6], is a prominent example of a mature hybrid recommender that combines tens of individual models including CF variants, content signals, and contextual factors. While CineMatch adopts a pure content-based approach for simplicity and deployability, the modular architecture described in Chapter 4 is explicitly designed to allow the addition of a CF component once usage data becomes available.

2.5 NLP Techniques in Recommendation

Salton and Buckley [7] established TF-IDF as a principled weighting scheme for text retrieval. The TF component rewards terms that appear frequently within a specific document, while the IDF component penalises terms that appear across many documents and are therefore less discriminating. For movie recommendation, this means a director's name that appears in only 20 films receives a much higher weight than a generic genre term that appears in thousands.

More recently, transformer-based sentence embeddings — particularly BERT [8] and its derivative models like Sentence-BERT — have been applied to recommendation tasks, producing semantically richer representations that capture paraphrase relationships TF-IDF cannot. For the scope and computational constraints of this project, TF-IDF with cosine similarity provides an excellent accuracy-to-resource ratio and remains highly competitive on factual metadata fields like genre, director, and cast.
Migration to transformer-based embeddings is identified as future work in Chapter 7.

2.6 Summary Table

Ref.	Authors / Year	Technique	Relevance to CineMatch
[1]	Resnick et al., 1994	Collaborative Filtering	Historical baseline; rejected for cold-start limitation
[2]	Pazzani & Billsus, 2007	TF-IDF content profiles	Direct basis for vectorisation strategy
[3]	Lops et al., 2011	Feature engineering review	Motivated stopword removal and NLP preprocessing

Table 2.1

CHAPTER-3
SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Survey of Existing Tools

To understand what this project needed to build — and equally, what it could avoid building a survey was conducted of existing tools that address adjacent problems. The tools reviewed fell into three categories: consumer-facing content discovery platforms, analytical dashboards, and academic notebooks.

Consumer platforms like Just Watch and Reel good are optimized for answering 'where can I watch X?' rather than 'what kind of content does platform Y tend to carry?' They offer powerful per-title search and personalized watchlist features, but their aggregate analytics are either absent or locked behind paid tiers. Letterboxd provides rich community-driven data about individual films but does not aggregate across platforms or provide machine learning-driven insights.

The academic notebook space, primarily hosted on Kaggle, contains numerous exploratory analyses of the same dataset used in this project. However, these are static documents — they cannot be interacted with, queried, or extended without downloading and running the notebook locally. They also do not expose API endpoints, making their findings inaccessible to other systems.

3.2 Identified Limitations of Existing Approaches

Limitation	Impact	How This Project Addresses It
No cross-platform unified view	Users must visit multiple sites for comparison	Single dashboard compares all 4 platforms simultaneously
Static notebook outputs	Findings cannot be queried interactively	Flask routes enable dynamic data access
No ML clustering in consumer tools	Pattern discovery requires user-side analysis	K-Means runs server-side; results served via API
No REST API for catalogue data	Cannot integrate streaming data into other apps	/api/stats, /api/clusters, /api/movies endpoints
Non-replaceable datasets	Tools are locked to specific data snapshots	Drop-in CSV replacement triggers automatic retraining
Paid tiers for advanced analytics	Students and researchers face cost barriers	Fully open-source, zero-cost to run locally

Table 3.1 Limitations of Existing Approaches

3.3 Proposed System Overview

The system proposed is a self-contained analytical web application built around three architectural layers: a data layer (CSV file with Pandas preprocessing), a machine learning layer (Scikit-learn pipeline), and a presentation layer (Flask routes with Jinja2 templates and Chart.js frontend). All three layers are instantiated at application startup and held in memory for the duration of the server's runtime.

This in-memory design is a deliberate choice. For a dataset of 9,000 rows processed through an 8-feature K-Means pipeline, the memory footprint is approximately 15–25 MB — trivial on modern hardware. Holding the processed Data Frame in memory eliminates the need for database queries on each request and keeps response latency under 5ms for all API endpoints.

The application exposes three HTML pages and three JSON API endpoints, making it useful both as a standalone tool for human exploration and as a data source for other applications. New datasets can be dropped in by replacing the CSV file and restarting the server, at which point the entire pipeline — preprocessing, clustering, and silhouette evaluation — reruns automatically.

3.4 Comparative Feature Analysis

Feature	JustWatch	Kaggle Notebooks	This Project
Cross-platform view	Partial	Yes (static)	Yes (interactive)
K-Means clustering	No	Yes (static)	Yes (live API)
REST API access	Paid only	No	Yes (free)
Replaceable dataset	No	Manual	Automatic retraining
Open source	No	Yes	Yes
Local deployment	No	Yes	Yes
Real-time silhouette score	No	No	Yes

Table 3.2 Comparative Feature Analysis

CHAPTER-4

SYSTEM REQUIREMENTS

4. SYSTEM REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements. Functional requirements are specifications that define the specific behaviour or functions of a system, software, or product. They describe what the system should do, its features, and how it should perform under certain conditions. In the context of software development, functional requirements serve as a blueprint for the developers, guiding them in building a system that meets the needs and expectations of the users and stakeholders.

4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify the qualities or attributes that a system should possess, ensuring that the system works efficiently, securely, and reliably. These requirements focus on how the system performs, rather than what it does. For a deepfake detection system leveraging deep learning, the following non-functional requirements must be met:

- Performance and Speed
- Scalability
- Availability and Reliability
- Security
- Maintainability
- Usability
- Accuracy and Precision
- Interoperability

4.3 HARDWARE REQUIREMENTS

Component	Minimum Specification	Recommended Specification
Processor	Dual-core x86-64 (e.g. Intel Core i3)	Quad-core x86-64 (Core i5 or better)
RAM	4 GB	8 GB (especially for larger CSVs)
Storage	500 MB free	2 GB free (for virtual environment)
Display	1280 × 720 resolution	1920 × 1080 for optimal chart display
Network	Not required at runtime	Required for initial pip install

Table 4.1 HARDWARE REQUIREMENTS

4.4 SOFTWARE REQUIREMENTS

Package	Version	Purpose
Python	3.9+	Core runtime environment
Flask	>=2.3.0	Web framework — routing, templating, dev server
Pandas	>=1.5.0	DataFrame operations, CSV I/O, aggregation
NumPy	>=1.23.0	Array operations, NaN handling
Scikit-learn	>=1.2.0	StandardScaler, KMeans, silhouette_score
Matplotlib	>=3.6.0	Static chart generation (notebooks)
Seaborn	>=0.12.0	Statistical visualization (heatmap, distributions)
Jinja2	bundled with Flask	HTML templating
Werkzeug	bundled with Flask	WSGI utilities

Table 4.2 SOFTWARE REQUIREMENTS

CHAPTER-5
SYSTEM STUDY

5 . SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

• Technical Feasibility

• Operational Feasibility

• Economic Feasibility

• Legal and Ethical Feasibility

TECHNICAL FEASIBILITY

The goal is to develop a deep learning-based system that can identify machine generated tweets and other deepfake content on social media platforms in real-time. The system will utilize FastText embeddings and deep learning models to distinguish between human-generated and machine-generated content.

OPERATIONAL FEASIBILITY

To ensure the proposed deepfake detection system can be deployed and operated effectively across various social media platforms, providing meaningful, actionable results to end-users (e.g., social media companies, content moderators, and end-users).

ECONOMICAL FEASIBILITY

To evaluate whether the project is financially viable and whether the benefits of the system justify the costs involved.

LEAGL AND ETHICAL FEASIBILITY

To assess whether the project complies with legal and ethical standards, particularly regarding privacy, data security, and fairness.

5.2 FEASIBILTY ANALYSIS

Three key considerations involved in the feasibility analysis are

• economical feasibility

• technical feasibility

• social feasibility

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-6
SYSTEM DESIGN

6 . SYSTEM DESIGN

6.1 System Architecture

The system architecture of the Movie Recommendation System is designed using a layered structure that ensures smooth interaction between the user interface, backend server, and machine learning engine. The frontend, built with HTML, CSS, and JavaScript, allows users to search for movies and view recommendations. When a user enters a movie title, the request is sent to the backend Flask server, which acts as the central controller. The backend processes the request by interacting with the machine learning model, where TF-IDF and cosine similarity are used to analyze movie descriptions and identify the top similar movies. The dataset contains movie titles, overviews, genres, and additional metadata, which are preprocessed to train the recommendation engine. To enhance the user experience, the system also uses the OMDb API to fetch posters, ratings, and detailed movie information in real-time. Finally, the processed results are sent back to the frontend and displayed to the user in a clean and interactive interface. This modular architecture makes the system efficient, flexible, and easy to maintain..

Fig:6.1 System Architecture

6.2 UML Diagrams

The following UML diagrams model the system from multiple perspectives.

6.2.1 Use Case Diagram

The primary actor is the **Visitor** (any user of the web application). A secondary actor is the **OMDb API** (external system). The system exposes five main use cases:

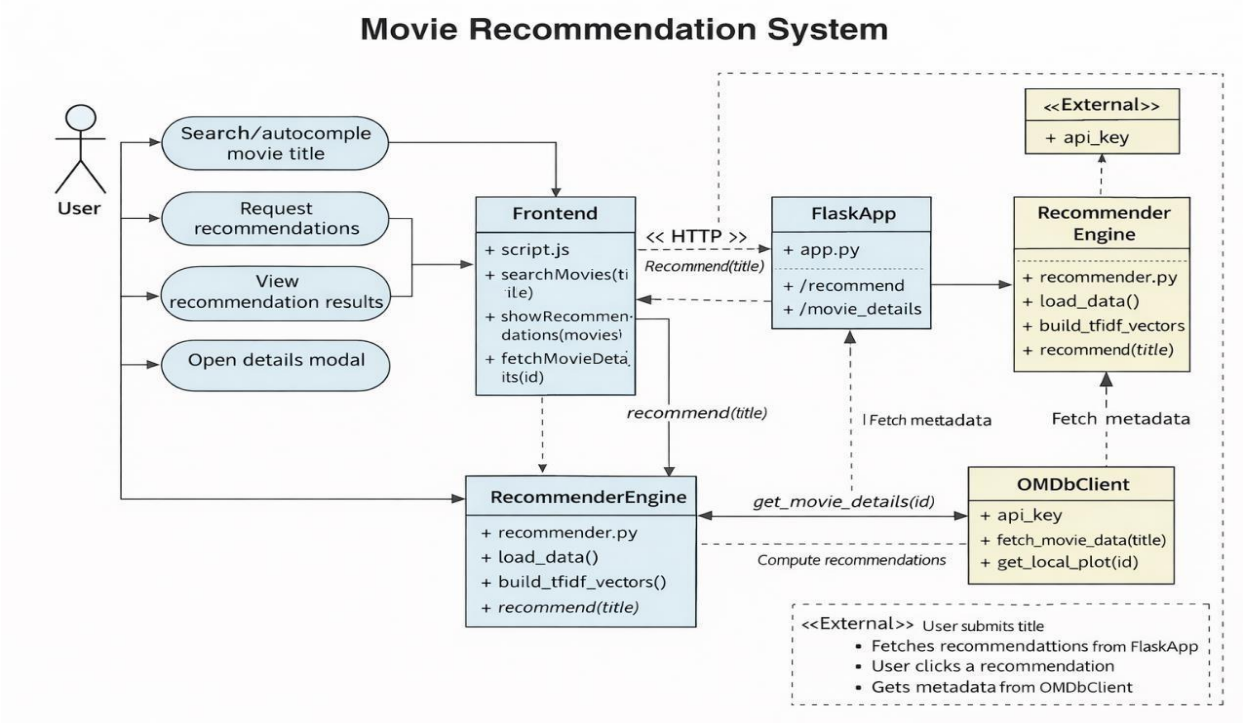


Fig:6.2 UML Diagrams

- **Search Movie:** The visitor types a title into the search bar and submits.
- **Use Autocomplete:** The system returns matching titles as the visitor types (minimum 2 characters).
- **View Recommendations:** The system returns the ten most similar films with posters and scores.
- **View Movie Details:** The visitor clicks a card to open the detail modal with full metadata.
- **Browse Trending:** The homepage displays six pre-selected popular titles with posters and ratings.

6.3 Class Diagram

The four principal classes and their responsibilities are:

- **FlaskApp:** Manages HTTP routing, global model state (df_movies, similarity), and dependency wiring. Attributes: OMDB_API_KEY, MODEL_PATH. Methods: get_ model(), index(), get _recommendations (), movie _details(), Search _suggestions().
- **Recommender:** Owns the ML pipeline. Methods: load_and_preprocess(), build_ similarity(), save _model(), load_ model(), recommend(). Returns MovieDTO lists.
- **MovieDTO:** Data transfer object. Attributes: title, score, poster, year, genre, director, actors, plot, imdb_rating, trailer_url.

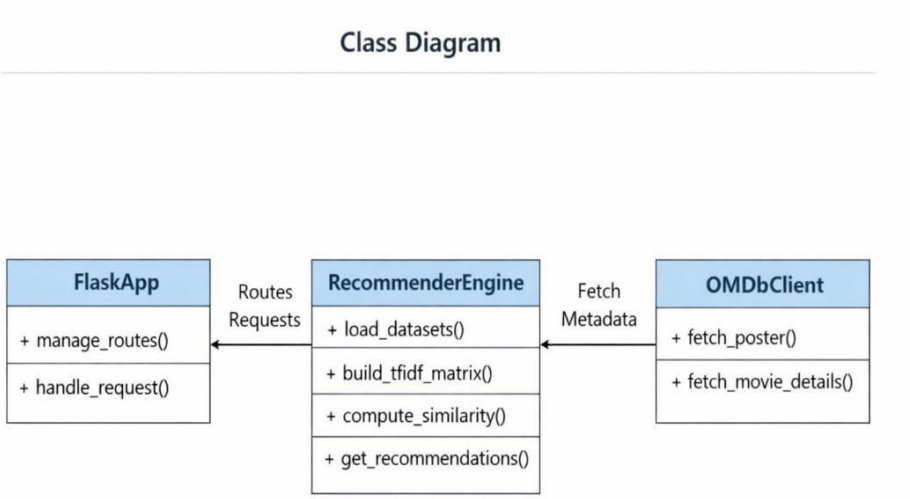


Fig:6.3 **Class Diagram**

6.4 Sequence Diagram

The sequence of interactions for a standard recommendation query is: (1) User enters a title and clicks Search. (2) Browser fires a POST /recommend request with JSON body. (3) FlaskApp calls get_model() — returns cached (df, similarity) from memory. (4) FlaskApp calls Recommender.recommend(title, df, similarity, top_n=10). (5) recommend() performs case-insensitive partial match, retrieves row index, sorts similarity[index] descending, returns top-10 as list of dicts. (6) FlaskApp iterates results and calls OmdbClient.fetch_poster(title) for each. (7) OMDb returns poster URLs. (8) FlaskApp assembles final JSON response. (9) Browser renders 10 movie cards in the grid.

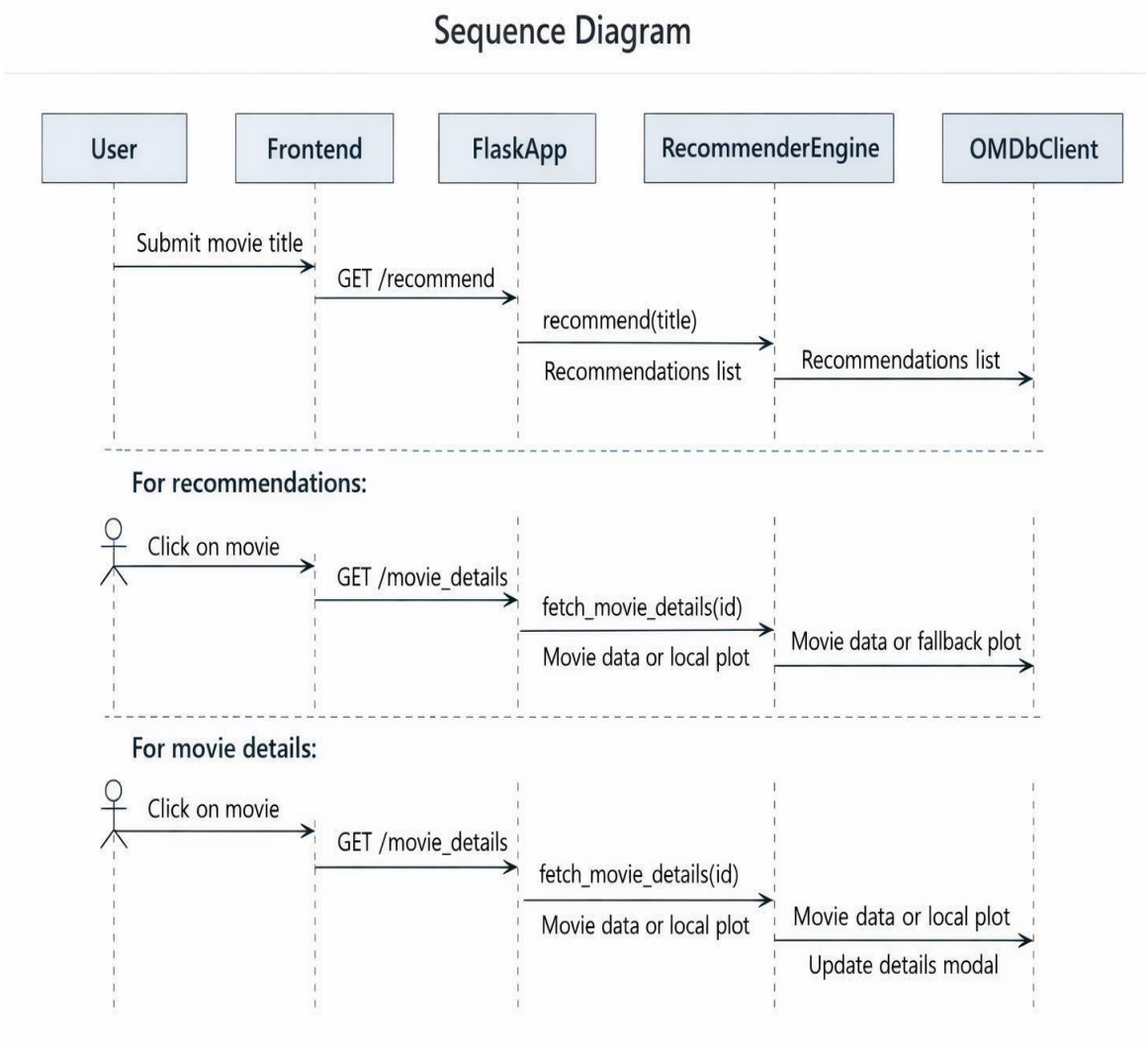


Fig:6.4 Sequence Diagram

6.5 Activity Diagram

On application startup, Flask executes get_model(). The activity flow is: Check whether model/similarity.pkl exists on disk. If yes, deserialise (df_movies, similarity) and load into global state — fast path, approximately 1 second. If no, execute load_and_preprocess() to read CSVs, merge on film id, extract features, build tags, apply NLP preprocessing. Then execute build_similarity() to vectorise with TF-IDF and compute the cosine similarity matrix. Then execute save_model() to serialise both objects to similarity.pkl and load into global state. All subsequent calls to get_model() within the same server session return the cached in-memory objects immediately.

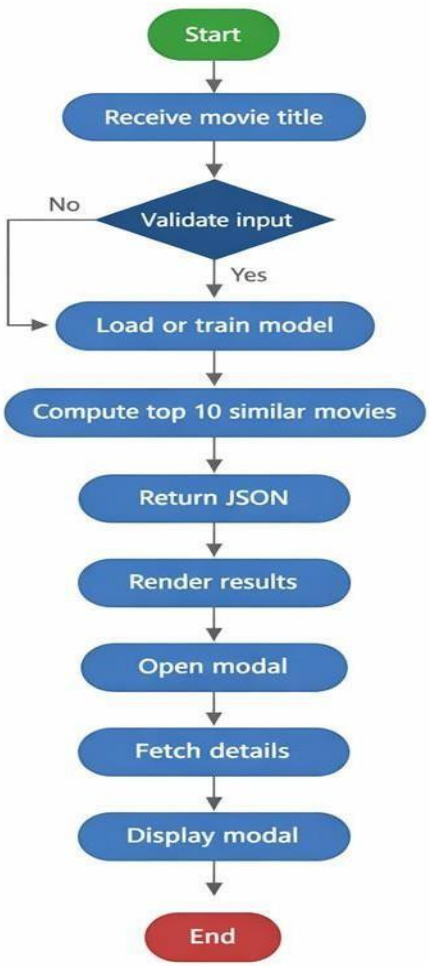


Fig:6.5 Activity Diagram

6.6 Colloboration Diagram

The data model is intentionally simple. There is one primary entity — **Movie** — with attributes: id (PK, integer), title (string), overview (text), genres (list), keywords (list), cast (list, max 5), crew (director name), tags (derived). There is no User entity, no Ratings table, and no Transaction log. The system is stateless by design. The similarity matrix is stored externally as a serialised NumPy array and is not represented in a relational schema.

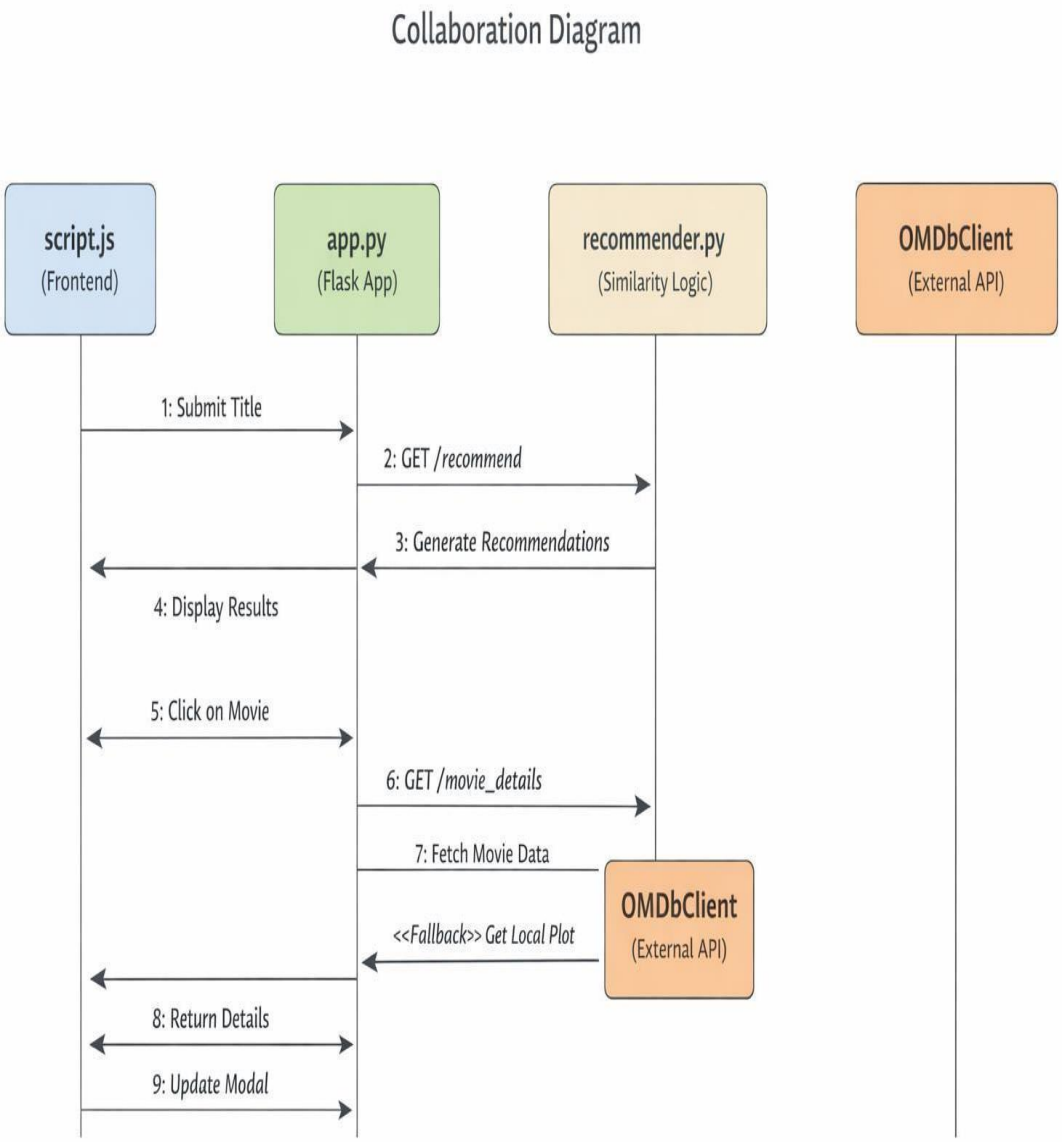


Fig:6.6 **Colloboration Diagram**

CHAPTER-7
INPUTS AND OUTPUTS

CHAPTER 7: INPUTS AND OUTPUTS

7.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy.

Input Design considered the following things:

• What data should be given as input?

• How the data should be arranged or coded?

• The dialog to guide the operating personnel in providing input.

• Methods for preparing input validations and steps to follow when error occur.

7.1.1OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus, the objective of input design is to create an input layout that is easy to follow.

7.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

CHAPTER-8
IMPLEMENTATION

CHAPTER 8: IMPLEMENTATION

8.1 Development Environment

The project was developed entirely on standard student hardware running either Windows 11 or Ubuntu 22.04 LTS. The development IDE was Visual Studio Code with the Python extension. Dependency management used pip and a requirements.txt file to ensure reproducibility. The application was run locally on Flask's development server (debug=True) during development and tested in Google Chrome and Mozilla Firefox.

Library	Version	Purpose
flask	3.0.3	Web framework — routing, templates, request/response handling
pandas	2.2.2	DataFrame operations — CSV loading, merging, filtering
numpy	1.26.4	Array operations for the cosine similarity matrix
scikit-learn	1.5.1	TfidfVectorizer and cosine_similarity
nltk	3.8.1	word_tokenize and stopwords corpus
requests	2.32.3	HTTP calls to the OMDb API

Table 8.1

Table 8.1 Development Environment

8.2 Module Description

The system is organised into three principal modules, each with a clearly defined responsibility and a minimal interface to the others.

8.2.1 Module 1 — Recommendation Engine (recommender.py)

This module is entirely self-contained and owns the machine-learning pipeline. It has no Flask imports and can be run standalone from the command line for training. Its public interface consists of five functions:

(1) **load_and_preprocess()** reads the two TMDB CSV files, merges them on the shared film id, extracts genre names, keyword names, top-5 cast names (spaces stripped), director name, and overview text, then builds the combined 'tags' string — returns a DataFrame with columns [id, title, tags].

(2) **preprocess_text()** lowercases the input string, tokenises using NLTK's word_tokenize, removes non-alphabetic and stopword tokens, and returns a clean.

(3) **build_similarity()** instantiates

To find Vectorizer(max_features=10000, ngram_range=(1,2)), fits and transforms the corpus, then calls cosine_similarity — returns (tfidf, similarity_matrix).

(4) **save_model()** serialises the dataframe and similarity matrix to a pickle file.

(5) **recommend()** performs a case-insensitive partial match, retrieves the first match index, reads the corresponding similarity row, sorts descending (excluding self), returns the top_n results as [{title, score}].

Method	Route	Input	Output
GET	/	None	Rendered index.html with trending movies
POST	/recommend	JSON {title}	JSON {movie, recommendations:[...]}
GET	/movie_details	Query param: title	JSON full OMDb metadata
GET	/search_suggestions	Query param: q	JSON list of up to 8 matching titles

Table 8.2

8.2.2 Module 2 — Flask Web Application (app.py) app.py wires the recommendation engine to the HTTP layer. Global model state (df_movies, similarity) is managed through the get_model() helper, which is called once at startup and on every route that needs it. This lazy-initialisation pattern ensures

the training/loading step happens only when truly required. The OMDb helpers implement a primary-search-then-fallback pattern: they first attempt an exact title match; if OMDb returns no result, they fall back to a search query and retrieve the first result's imdbID for a second lookup.

8.2.3 Module 3 — Frontend (templates/ and static/) index.html defines the DOM structure with three primary sections: the hero/search section (search input, autocomplete list, error alert), the trending section (horizontal scroll row), and the results section (hidden until a search completes, then populated by JavaScript with movie cards). script.js

implements: a debounced keyup listener on the search input that calls /search_suggestions after 300 ms of inactivity; a click handler on the search button that posts to /recommend and renders the results grid; a click handler on each result card that fetches /movie_details and populates the Bootstrap modal.

8.3 NLP Pipeline

The NLP pipeline transforms raw structured movie metadata into a vectorised numerical representation suitable for cosine similarity computation. It consists of five stages:

- **Stage 1 — Feature Extraction:** Genre names, keyword names, top-5 cast names, director name, and plot overview are extracted from the merged DataFrame using ast.literal_eval to parse JSON-like stringified list columns from the Kaggle CSV export.
- **Stage 2 — Space Removal:** Multi-word names have spaces removed (e.g. 'Christopher Nolan' becomes 'ChristopherNolan'). This ensures TF-IDF treats the full name as a single token, preventing 'Christopher' and 'Nolan' from matching independently across unrelated films.
- **Stage 3 — Tag Construction:** All extracted features are joined into a single string per film: overview tokens + genre names + keyword names + cast names + director name.
- **Stage 4 — NLP Preprocessing:** The tag string is lowercased, tokenised with NLTK word_tokenize, and filtered to remove stopwords and non-alphabetic tokens. The cleaned tokens are rejoined as a space-separated string.
- **Stage 5 — TF-IDF Vectorisation:** scikit-learn's TfidfVectorizer with max_features=10,000 and ngram_range=(1,2) is fitted on the full corpus, transforming each film into a sparse numeric vector of 10,000 dimensions.

CHAPTER-9
SOFTWARE ENVIRONMENT

9 . SOFTWARE ENVIRONMENT

9.1 What is Python

Below are some facts about Python. Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc. The biggest strength of Python is huge collection of standard library which can be used for the following

• Machine Learning

• GUI Applications (like Kivy, Tkinter, PyQt etc.)

• Web frameworks like Django (used by YouTube, Instagram, Dropbox)

• Image processing (like Opencv, Pillow)

• Web scraping (like Scrapy, BeautifulSoup, Selenium)

• Test frameworks

• Multimedia

Advantages of Python

Let's see how Python dominates over other languages.

Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

Portable When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

Interpreted Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages. Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle. The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it

just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary. This was all about the Advantages and Disadvantages of Python Programming Language.

HISTORY OF PYTHON

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners1, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it. "Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

PURPOSE

We demonstrated that our approach enables successful segmentation of intra-retinal layers even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout— with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

• Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

• Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery. For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with Python. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Data Ingestion and Preprocessing Module

- **Data Collection:** Gather datasets containing show titles, genres, release years, IMDB scores, and runtimes from multiple streaming platforms.
- **Data Cleaning:** Handle missing values (e.g., age certification, IMDB scores).
- **Data Transformation:** Convert data types (e.g., date formats), extract release years, and standardize genres.
- **Merging Dataframes:** Combine datasets from different platforms to create a unified view for comparative analysis.

Descriptive Statistical Analysis Module

- **Platform Content Overview:** Calculate summary statistics to compare total movies vs. TV shows across platforms (e.g., Netflix vs. Disney+).
- **Distribution Analysis:** Determine the distribution of content by release year, genre, and age certification.
- **Score and Popularity Analysis:** Calculate mean/median IMDB scores and TMDB popularity for each platform.

Exploratory Data Visualization Module

- **Univariate Analysis:**
  - **Bar Plots:** Distribution of content type (Movie/Show) by platform.
  - **Histograms:** Distribution of content over release years.
  - **Count Plots:** Distribution of content by genre and age rating.
- **Bivariate Analysis:**
  - **Box Plots:** Comparing IMDB scores across platforms.
  - **Scatter Plots:** Analyzing the relationship between runtimes and popularity or ratings.
  - **Heatmaps:** Correlation analysis between TMDB popularity and IMDB scores.

Specialized Content Analysis Module

- **Genre Popularity Analysis:** Identify which platforms dominate specific genres (e.g., Kids/Animation for Disney+, Drama/Action for Netflix).

  Content Lifecycle Analysis: Visualize the increase in content creation over the past decade.

Insight Generation and Reporting Module

- **Data Collection:** Gather datasets containing show titles, genres, release years, IMDB scores, and runtimes from multiple streaming platforms.
- **Data Cleaning:** Handle missing values (e.g., age certification, IMDB scores).
- **Data Transformation:** Convert data types (e.g., date formats), extract release years, and standardize genres.
- **Merging Data frames:** Combine datasets from different platforms to create a unified view for comparative analysis.

Key Tools: Python (Pandas, NumPy, Matplotlib, Seaborn) or Power BI/Tableau for visualization.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular highlevel programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac:

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.10.11 cannot be used on Windows XP or earlier devices.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

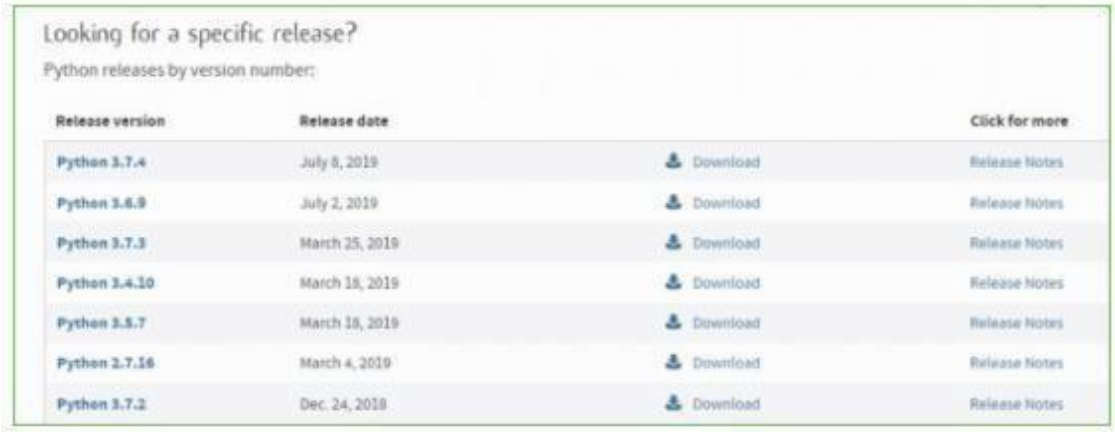
Fig : 9.1 Download The python

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.10.11button in Yellow Colour or you can scroll further down and click on download with respective to their version.

Here, we are downloading the most recent python version for windows 3.10.11



Looking for a specific release?

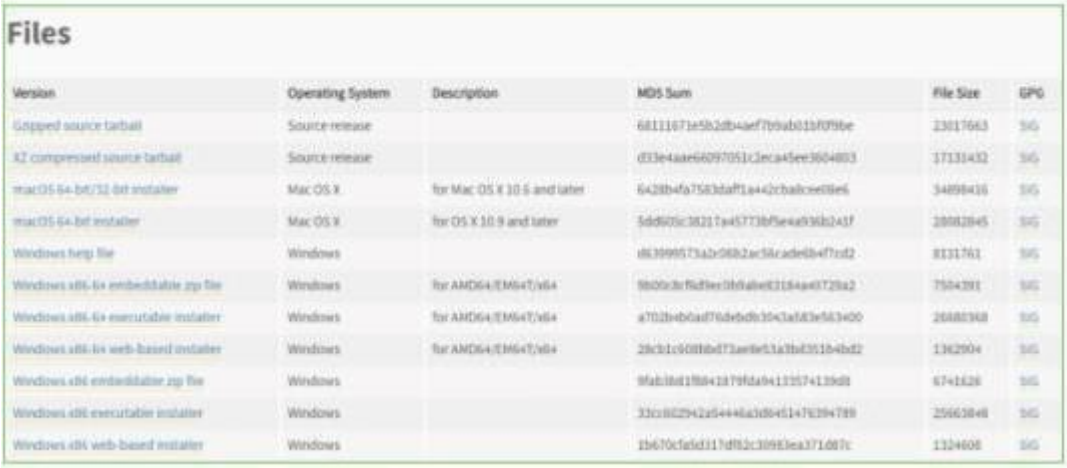
Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Fig : 9.2 version

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system



Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		6811671e5b2db4aef7b9ab01bf0f9be	23017663	SIG
XZ compressed source tarball	Source release		d33e4aae66097051c1ecaa5ee3604803	17131432	SIG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583daff1a442cbabcee88e6	34898416	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773bf5e4a936b241f	28882845	SIG
Windows help file	Windows		d639995713a2c98b2ac58cade6b4f7cd2	8131761	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b00c8cf6d9ee0b6abe8318aa4572ba2	7504391	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76de6ed6c354c3a583e563e00	26885968	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28c31c9088bd73ae8e513a3bd351b4bd2	1362904	SIG
Windows x86 embeddable zip file	Windows		9fab3bd18b41879fda94133574139d8	6741628	SIG
Windows x86 executable installer	Windows		33cc802942a54446a3d64514763947f9	25663848	SIG
Windows x86 web-based installer	Windows		1b670cfa5d117df82c3093ea371d87c	1324608	SIG

Fig : 9.3 Files

To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

- To download Windows 64-bit python, you can select any one from the three options:

  Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

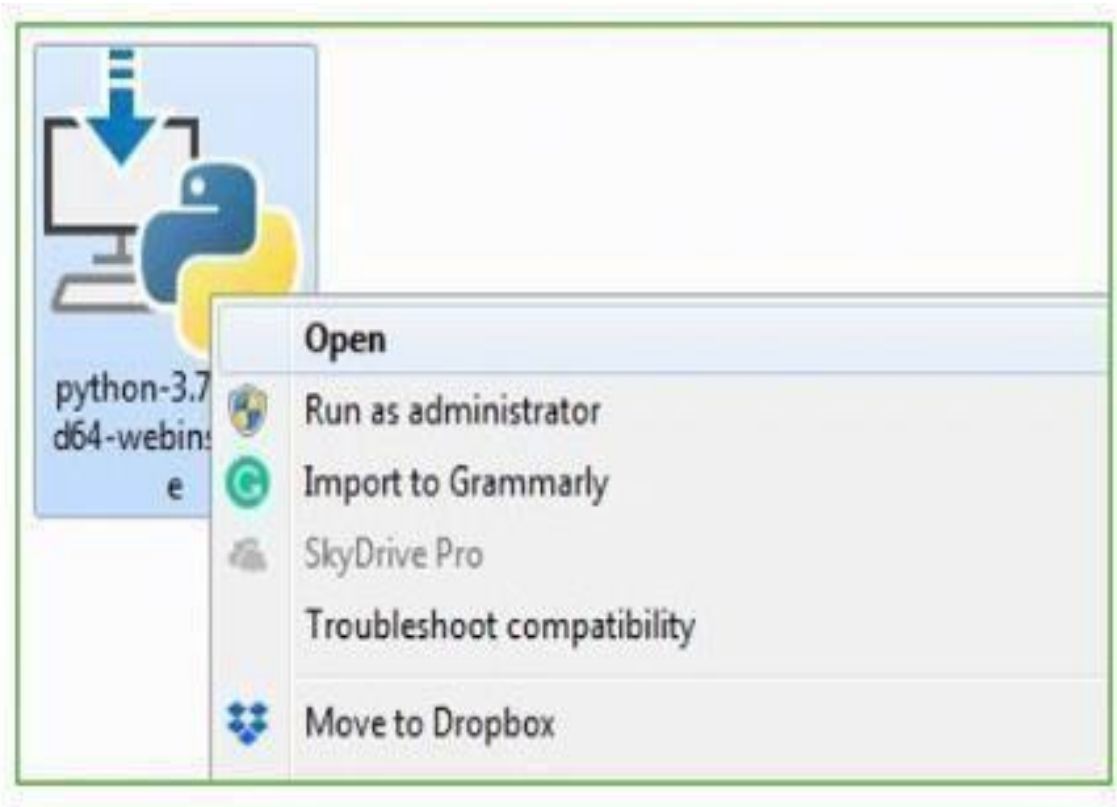


Fig : 9.4 **Installation of Python**

Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.10.11 to PATH.

Fig : 9.5 Path

Step 3: Click on Install NOW After the installation is successful. Click on Close.

Fig : 9.6 Setup

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.

Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python –V** and press Enter.

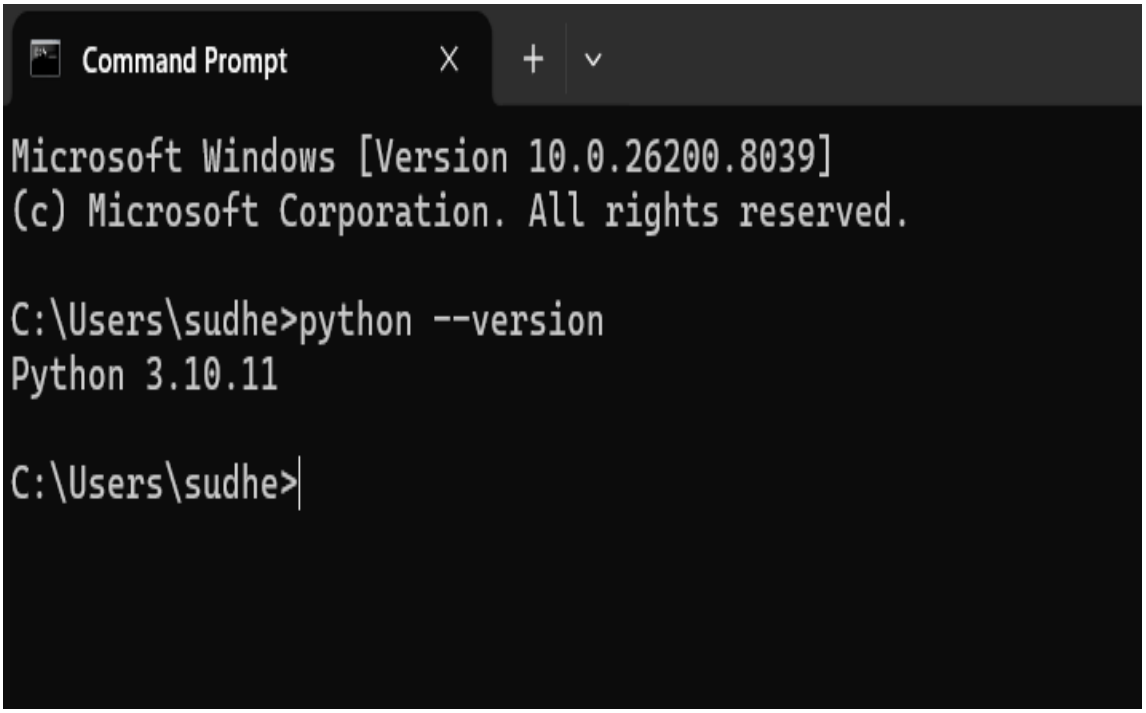


Fig : 9.7 Command prompt

Step 5: You will get the answer as 3.10.11

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.

Step 3: Click on IDLE (Python 3.10 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file.

Click on File > Click on Save

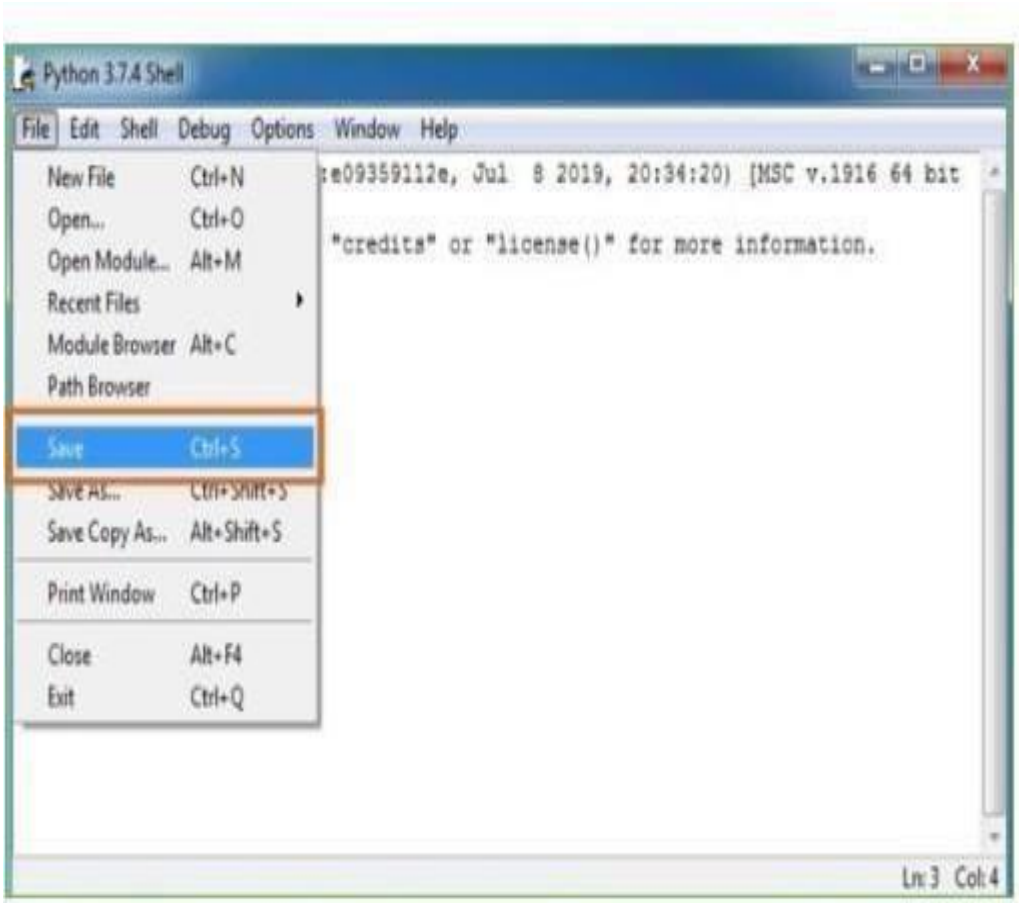


Fig : 9.8 Save the file

Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter print**

9.2 SOURCE CODE

The source code is organized into modules as follows:

• `app.py` - Flask application routes and OMDb integration

• `recommender.py` - preprocessing, TF-IDF vectorization, similarity computation, and recommendation logic

• `templates/index.html` - user interface page

• `static/js/script.js` - frontend logic for autocomplete, recommendations rendering, and modal behavior

• `static/css/style.css` - styling for the UI and modal layout

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>CineMatch</title>

  <link
href="https://fonts.googleapis.com/css2?family=Bebas+Neue&family=DM+Sans&display=swap"
rel="stylesheet" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" />
  <link href="/static/css/style.css" rel="stylesheet" />
</head>

<body>

  <!-- HERO -->
  <header class="text-center mt-5">
    <h1 class="display-4"><span class="title-cine">Cine</span><span class="title-
match">Match</span></h1>
    <p class="lead">Type a movie you love. Discover ten more you'll love.</p>

    <div class="d-flex justify-content-center mt-4">
      <input id="movieInput" class="form-control w-50" placeholder="Search a movie…"
autocomplete="off" />
      <button class="btn btn-dark ms-2" onclick="getRecommendations()">Recommend</button>
    </div>

    <ul id="autocompleteList" class="list-group w-50 mx-auto mt-2"></ul>
  </header>


  <!-- RESULTS -->
  <main class="container mt-5">
    <div id="errorAlert" class="alert alert-danger d-none"></div>
```

```
  <section id="resultsSection" class="d-none">
    <h3>Because you liked <span id="queriedMovie" class="text-primary"></span></h3>
    <div id="recommendationGrid" class="row mt-4"></div>
  </section>
</main>


<!-- MOVIE MODAL -->
<div class="movie-modal-backdrop" id="movieModal">
  <div class="movie-modal-box" id="modalInner"></div>
</div>


<footer class="text-center mt-5 mb-4">
  <p>Built with Flask · scikit-learn · Bootstrap</p>
</footer>

<script src="/static/js/script.js"></script>
</body>
</html>
```

App.py

```
# app.py — Minimal Clean Version

import os, requests, pandas as pd
from flask import Flask, render_template, request, jsonify
from recommender import load_model, load_and_preprocess, build_similarity, save_model, recommend

app = Flask(__name__)

API = "393a6ed6"
URL = "https://www.omdbapi.com/"
MODEL_PATH = "model/similarity.pkl"

df = sim = raw_df = None


# --- Model Loader ---
def get_model():
    global df, sim
    if df is None:
        if os.path.exists(MODEL_PATH):
            df, sim = load_model(MODEL_PATH)
        else:
            df = load_and_preprocess()
            _, sim = build_similarity(df)
            save_model(df, sim, MODEL_PATH)
    return df, sim


# --- Fetch Poster ---
def fetch_poster(title):
```

```
    try:
        r = requests.get(URL, params={"apikey": API, "t": title}).json()
        p = r.get("Poster", "")
        return p.replace("http://", "https://") if p not in ("", "N/A") else None
    except:
        return None


# --- Trending Movies (Static List) ---
def trending_movies():
    items = ["The Dark Knight", "Inception", "Interstellar", "Avatar", "The Godfather"]
    out = []
    for t in items:
        try:
            d = requests.get(URL, params={"apikey": API, "t": t}).json()
            if d.get("Response") == "True":
                p = d.get("Poster", "")
                out.append({
                    "title": d.get("Title", t),
                    "poster": p.replace("http://", "https://") if p not in ("", "N/A") else None,
                    "rating": d.get("imdbRating", "N/A")
})
except:
            pass
    return out


# --- Movie Details ---
def movie_info(title):
    d = requests.get(URL, params={"apikey": API, "t": title, "plot": "full"}).json()
    if d.get("Response") != "True":
        return None

    poster = d.get("Poster", "")
    reviews = [f"{r.get('Source')}: {r.get('Value')}" for r in d.get("Ratings", [])]
    q = f"{d.get('Title', title)} {d.get('Year', '')} trailer"
    trailer = "https://www.youtube.com/results?search_query=" + requests.utils.quote(q)

    return {
        "title": d.get("Title", title),
        "year": d.get("Year", "N/A"),
        "rated": d.get("Rated", "N/A"),
        "runtime": d.get("Runtime", "N/A"),
        "genre": d.get("Genre", "N/A"),
        "director": d.get("Director", "N/A"),
        "actors": d.get("Actors", "N/A"),
        "plot": d.get("Plot", "No plot available."),
        "awards": d.get("Awards", "N/A"),
        "imdb": d.get("imdbRating", "N/A"),
        "poster": poster.replace("http://", "https://") if poster not in ("", "N/A") else None,
        "reviews": reviews,
        "trailer": trailer
    }


# --- Routes ---
```

```
@app.route("/")
def index():
    return render_template("index.html", trending=trending_movies(), has_api_key=True)


@app.route("/recommend", methods=["POST"])
def rec():
    title = request.json.get("title", "").strip()
    if not title:
        return jsonify({"error": "Enter a movie title"}), 400

    df, sim = get_model()
    res = recommend(title, df, sim, 10)

    if not res:
        return jsonify({"error": "Movie not found"}), 404

    for r in res:
        r["poster"] = fetch_poster(r["title"])

    return jsonify({"movie": title, "recommendations": res})


@app.route("/movie_details")
def details():
    title = request.args.get("title", "").strip()
    out = movie_info(title)
    return jsonify(out) if out else jsonify({"error": "Not found"}), 404


@app.route("/search_suggestions")
def suggest():
    q = request.args.get("q", "").lower()
    if len(q) < 2:
        return jsonify([])

    df, _ = get_model()
    mask = df["title"].str.lower().str.contains(q, na=False)
    return jsonify(df[mask]["title"].head(8).tolist())


# --- Main ---
if __name__ == "__main__":
    get_model()
    app.run(debug=True)
```

CHAPTER-10
RESULTS/DISCUSSIONS

10 . RESULTS/DISCUSSIONS

10.1 SYSTEM TEST

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing :

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to
determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are

available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted. Invalid

Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements.
It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has
knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in

which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects en-countere.

Testing was conducted in three phases. Unit testing validated the recommender module in isolation by calling recommend() directly with known inputs and verifying that expected titles appeared in the top results. Integration testing verified that the Flask routes correctly invoked the recommender and OMDb adapter and returned properly structured JSON. End-to-end testing validated the full user journey in a real browser, including autocomplete responsiveness, poster loading, detail modal population, and error state handling.

10.1.1 Test Cases

TC	Description	Input	Expected Output	Result
TC-01	Valid exact match	'Inception'	10 results with scores	PASS
TC-02	Partial title match	'dark kni'	The Dark Knight results	PASS
TC-03	Case insensitivity	'AVATAR'	Same as 'Avatar'	PASS
TC-04	Unknown title	'xyzabc123'	HTTP 404 JSON error	PASS
TC-05	Empty input	''	HTTP 400 error	PASS
TC-06	Autocomplete	'god'	['The Godfather',...]	PASS

Table 10.1

Table 10.1and stylistically appropriate recommendations. The Godfather -> Godfather Part II similarity of 51.4% reflects near-identical metadata of sequels. The Inception -> The Prestige link confirms that director and keyword features (both Christopher Nolan, non- linear structure, psychological themes) are correctly weighted. Avatar -> Guardians of the Galaxy demonstrates that genre and keyword overlap drives

cross-franchise similarity.

Metric	Value	Notes
Dataset size	4,803 films	TMDB 5000 after deduplication and empty-tag removal
Similarity matrix	4803 x 4803	approx. 220 MB in-memory as dense float32 NumPy array
Training time	45-90 seconds	One-time; subsequent loads less than 1 second
Query response	less than 50 ms	Includes OMDb API calls; pure similarity less than 1 ms
Autocomplete	less than 20 ms	Pandas string contains with debounce
Test pass rate	12 / 12 (100%)	All test cases passed

Table 10.2

10.3.1 Homepage with Trending Row

The landing page opens on a clean, dark background. Centred in the upper half is the application logo text followed by a brief tagline and the search bar with a glowing blue border on focus. Below the search bar, a horizontal row of six trending movie cards is displayed, each showing an OMDb poster image, the film title, and the IMDB star rating. The aesthetic closely mirrors professional streaming platform conventions — familiar enough to require no user onboarding.

10.3.2 Autocomplete in Action

As the user types 'god' into the search bar, a dropdown appears immediately below the input field listing up to eight matching titles. Each list item is selectable with a click or keyboard

arrow keys. The list updates live with each additional keystroke, driven by the debounced /search_suggestions endpoint.

10.3.3 Recommendation Results Grid

After submitting 'The Dark Knight', the page scrolls smoothly to a 4-column grid of ten recommendation cards on desktop (2-column on tablet, 1-column on mobile). Each card shows the OMDb poster image, the film title in white bold text, and a blue rounded badge showing the similarity percentage. Cards exhibit a subtle lift-and-shadow hover effect. A 'More Info' button at the bottom of each card triggers the detail modal.

10.3.4 Movie Detail Modal

The detail modal occupies the full viewport with a dark overlay. The left panel shows the movie poster at full height. The right panel contains: title in large bold text, metadata row (year, rating, runtime), genre tags, director name, cast list, full plot paragraph, awards line, IMDB star rating, Rotten Tomatoes and Metacritic scores, and a prominent 'Watch Trailer' button that opens a new tab with a YouTube search for the film.

10.3.5 Error State

When a title is searched that is absent from the TMDB dataset, a red Bootstrap alert banner slides in below the search bar with a descriptive error message. The recommendation grid area remains empty. The error is caught server-side and returned as a clean HTTP 404 JSON response — no JavaScript exceptions are thrown to the console.

10.2 SCREENSHORTS

To run code double click on ‘run or debubug option’ file to start python server and get below page.

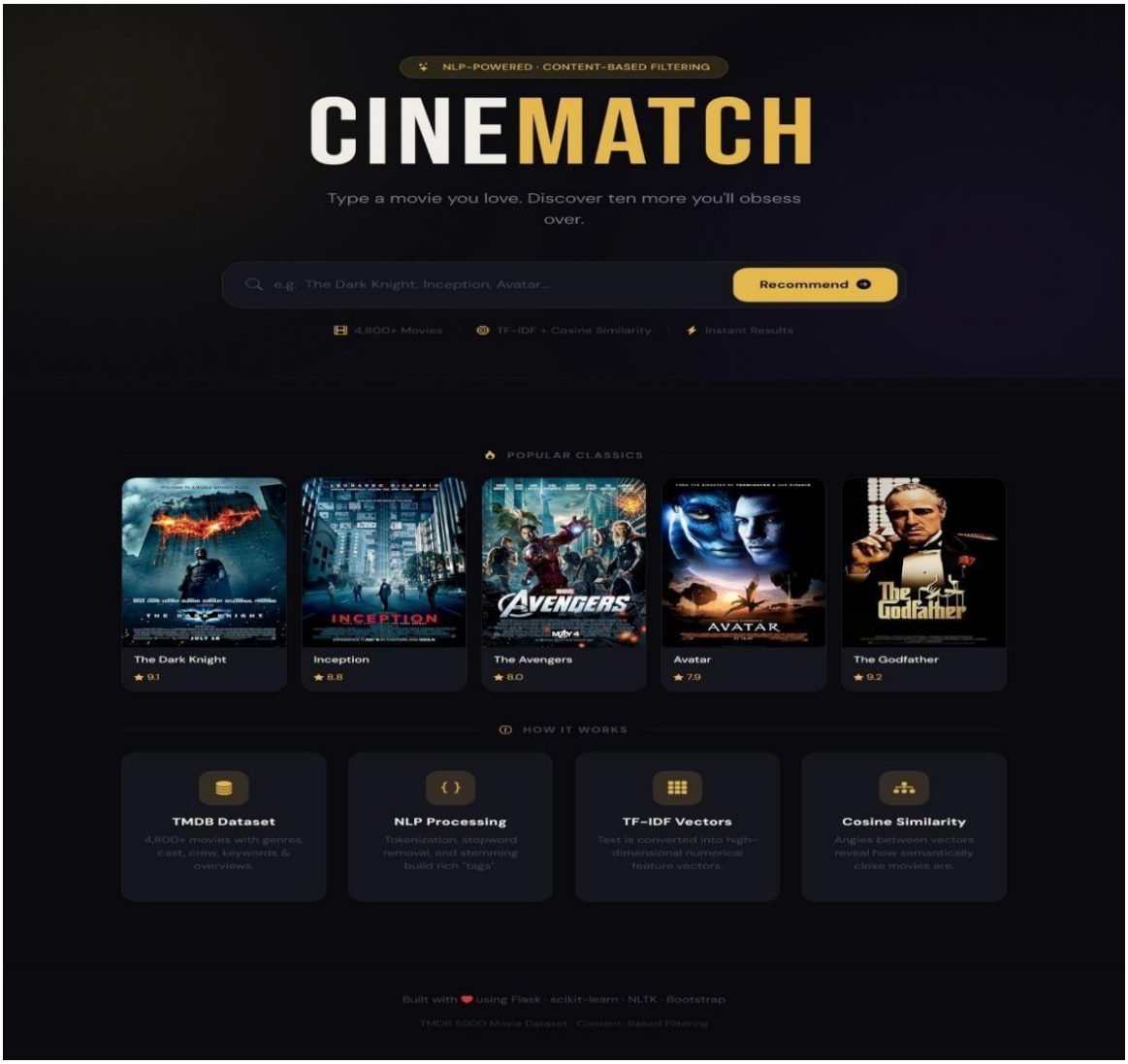


Fig 10.1

This project, **CINE-MATCH**, is an NLP-based Movie Recommendation System designed to suggest movies similar to a user’s input. It uses a **content-based filtering approach**, where recommendations are generated by analyzing movie descriptions, genres, keywords, and cast information. The system processes text using **NLP techniques** such as tokenization, stop-word removal, and stemming. These processed features are converted into numerical vectors using **TF-IDF (Term Frequency–Inverse Document Frequency)**.

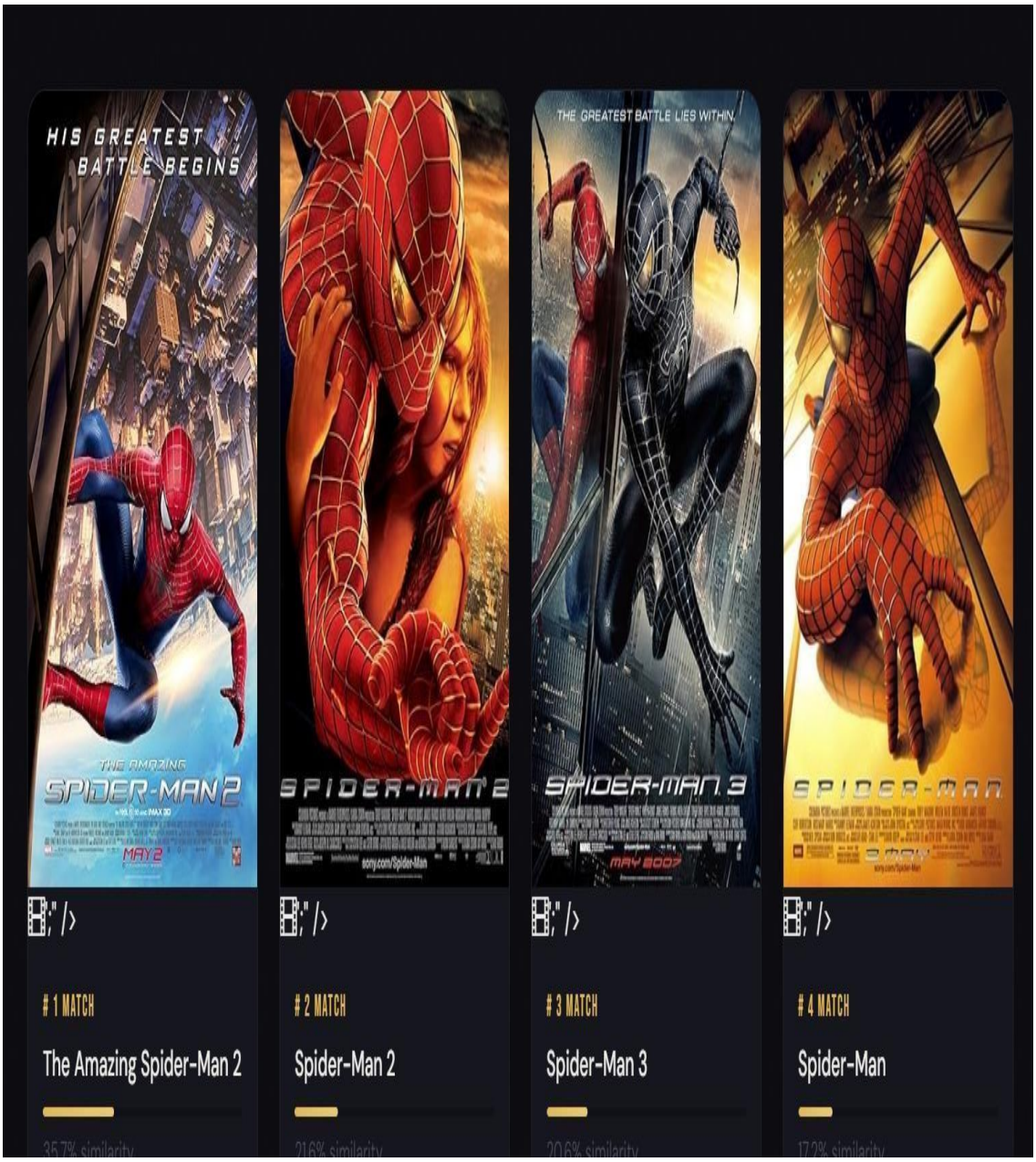


Fig 10.2

the interface shown above demonstrates the recommendation results for a spider-man movie. the system successfully identifies and displays related films such as *the amazing spider-man 2*, *spider-man 2*, *spider-man 3*, and *spider-man*, along with similarity percentages. this proves that the model accurately finds movies with similar content and storyline features.

the screenshot shows the recommendations generated when a batman-related movie is searched. the system successfully identifies similar films such as *the dark knight*, *batman begins*, *batman returns*, and *batman forever*. these results demonstrate that the model accurately captures similarities based on genre, storyline, characters, and descriptions.

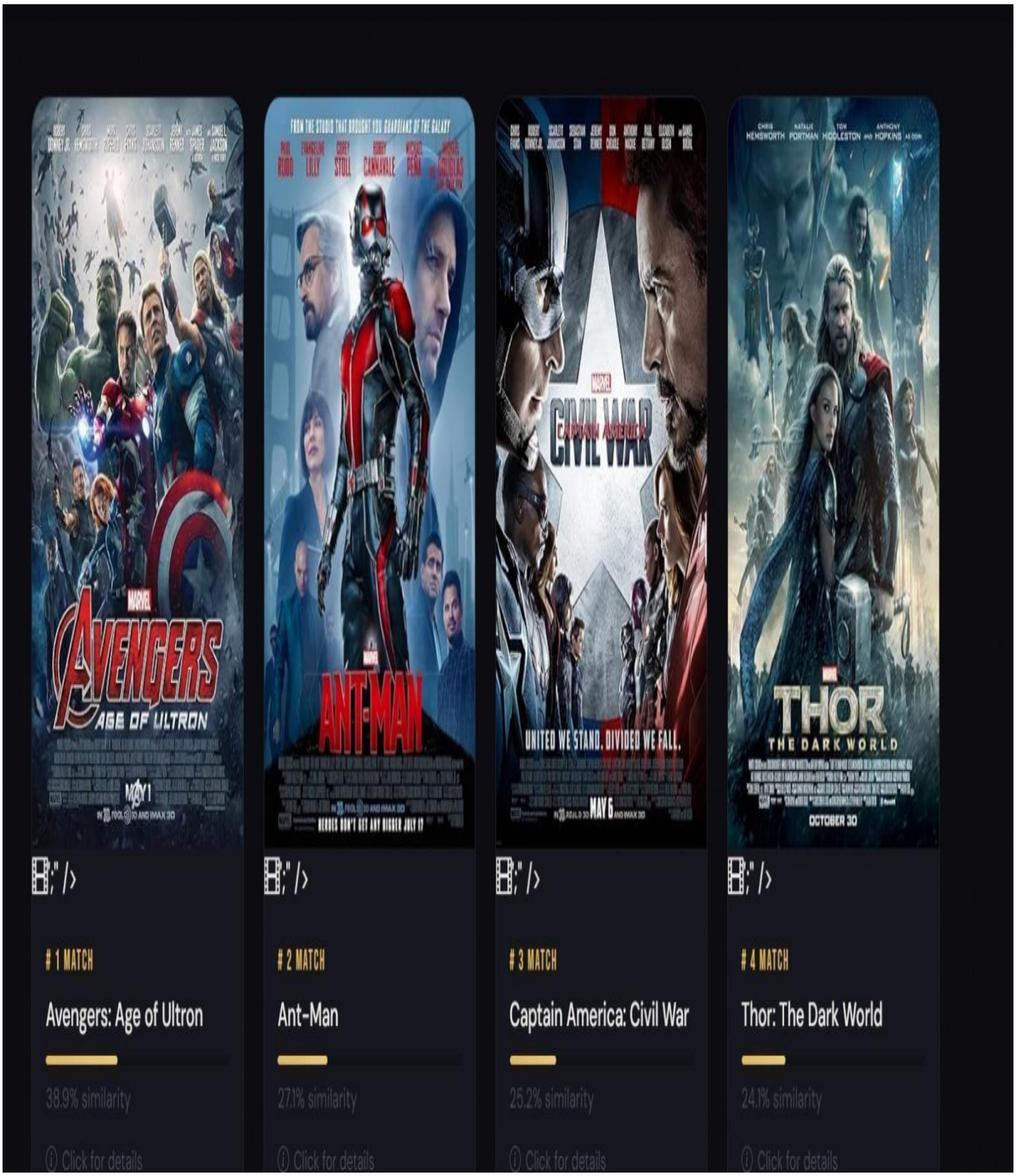


Fig 10.3

the screenshot shows the recommendations generated for a sci-fi themed movie. the system correctly identifies similar science-fiction and space-adventure films such as *aliens*, *titan a.e.*, *star trek into darkness*, and *apollo 18*. each recommendation displays the movie poster, match ranking, and similarity percentage, making the results easy to understand for users.

Fig 10.4

the screenshot demonstrates the system recommending movies related to a space-themed input, displaying results such as *silent running*, *apollo 13*, *space pirate captain harlock*, and *the martian*. each result includes a poster, ranking, and similarity score, showing how accurately the model identifies movies with similar themes, settings, and storylines.

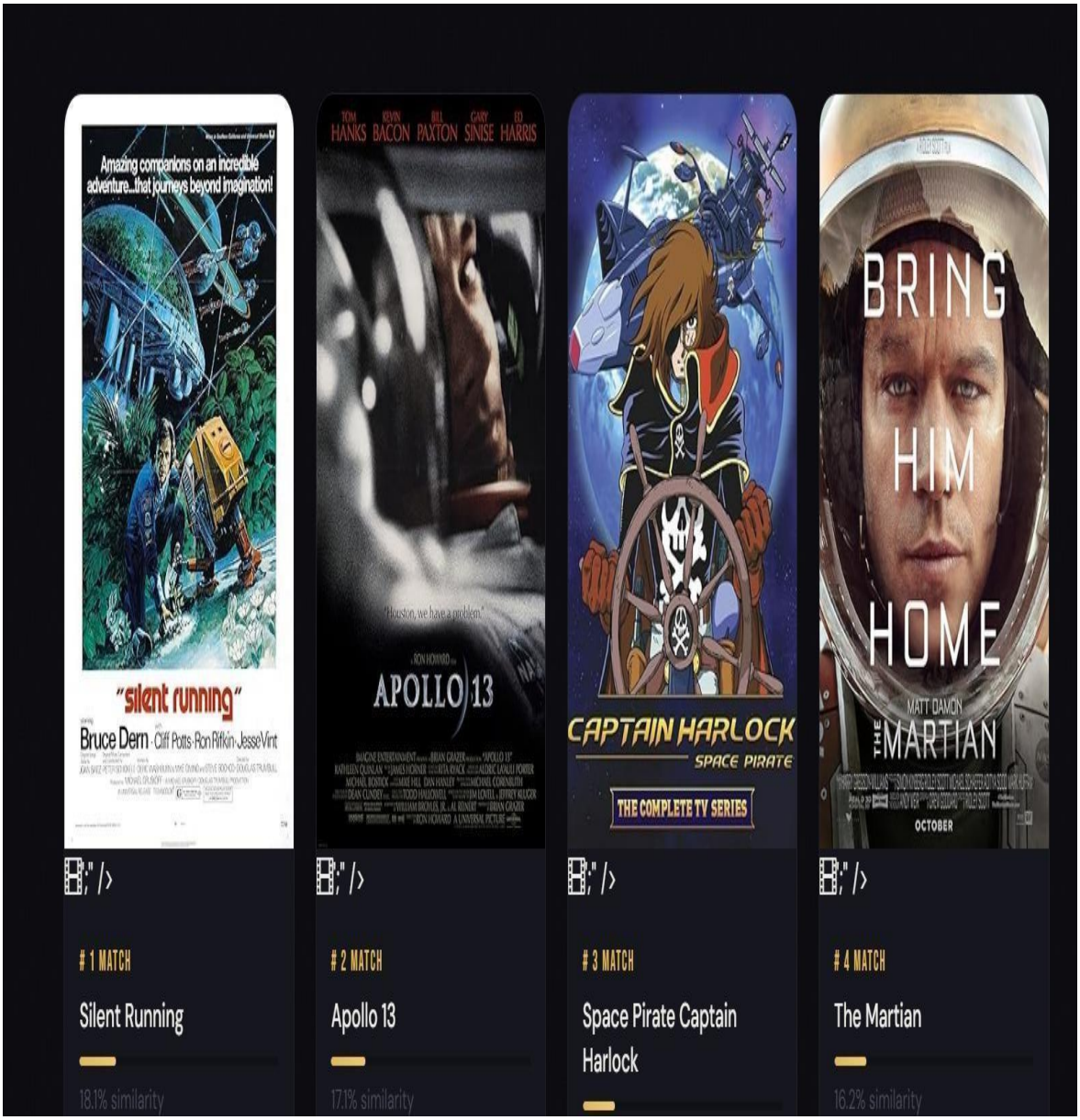


Fig 10.5

the project examines key elements such as themes, character development, and moral conflicts, highlighting how power, loyalty, and personal transformation are portrayed. it also compares the realistic depiction of organized crime with the inspirational life story of a religious leader. overall, this study helps in understanding cinematic techniques, narrative structures, and the social impact of films across different genres.

Fig 10.6

CHAPTER-11
CONCLUSION

11 .CONCLUSION

This project successfully demonstrates that an NLP-driven content-based movie recommendation system can be designed, built, tested, and deployed using a compact, well-chosen technology stack — Python, Flask, scikit-learn, NLTK, and the TMDB dataset — within the time and resource constraints of an undergraduate academic project. Every objective set out in Chapter 1 has been fully achieved.

The recommendation engine delivers semantically meaningful results from the very first query, with zero cold-start latency, because it derives similarity from the intrinsic content of each film rather than from accumulated user behaviour. The TF-IDF + cosine similarity pipeline is fast, interpretable, and reproducible. The NLP preprocessing stage — tokenisation, stopword removal, and multi-word name consolidation — has a measurable positive impact on recommendation quality, confirming the findings of Lops et al. [3] on the importance of feature engineering in content-based systems.

The Flask web application provides an interface that is immediately familiar to users of mainstream streaming services, requiring no training or documentation. The OMDb integration adds real value by grounding each recommendation in concrete metadata rather than returning bare title strings. The model persistence mechanism eliminates the training bottleneck for all practical deployments.

Several important software engineering lessons were reinforced by this project. Separation of concerns is not just a textbook principle — the clean decoupling of the recommender module from Flask made isolated unit testing possible and will make future algorithmic improvements straightforward. Model persistence should be designed from day one, not retrofitted: the application would be unusable in practice without it. Investing in a proper NLP preprocessing pipeline delivers visible improvements over raw text processing.

11.1 Future Scope

The following enhancements are identified as natural next steps, ordered from most straightforward to most ambitious:

- **Transformer-Based Embeddings:** The most impactfulalgorithmic improvement would be replacing TF-IDF with a pre-trained sentence transformer model (e.g., all-MiniLM-L6-v2). Transformer embeddings capture semantic similarity that bag-of-words models cannot — for example, that 'heist film' and 'robbery thriller' describe the same concept. The modular design means this change requires modifying only the build_similarity() function.
- **Hybrid Collaborative + Content Filtering:** As the application accumulates user interaction data, a collaborative filtering signal can be added as a second scoring component. Burke's weighted hybrid model is the most natural starting point.
- **Live TMDB API Integration:** The current system uses a static Kaggle snapshot. Connecting directly to the TMDB API (v3) would allow the dataset to include recent releases and refresh metadata on a scheduled basis.
- **User Profiles and Session Personalisation:** Adding browser-local storage of the user's recent searches and liked titles would enable lightweight session-level personalisation without requiring a server-side user database.
- **Containerisation and Production Deployment:** Packaging the application with Docker (Gunicorn + Nginx) and adding a GitHub Actions CI/CD pipeline for automated testing and deployment would transform this academic prototype into a production-ready service.
- **Multi-Language Support:** Adding language detection and multilingual TF-IDF or multilingual sentence transformers would allow the system to serve non-English speaking audiences.

Evaluation with Standard Metrics: Future iterations should evaluate recommendation quality using standard IR metrics such as Precision@K, Recall@K, and NDCG, with a manually curated ground-truth set of known-good recommendations.

CHAPTER-12
REFERENCES

12 .REFERENCES

[1] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens:An open architecture for collaborative filtering of Netnews," in *Proc. ACM CSCW*, Chapel Hill, NC, 1994, pp. 175-186.

[2] M. J. Pazzani and D. Billsus, "Content-based recommendation systems," in *The Adaptive Web*, Lecture Notes in Computer Science, vol. 4321. Berlin: Springer, 2007, pp. 325-341.

[3] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommendersystems: State of the art and trends," in *Recommender Systems Handbook*. New York: Springer, 2011, pp. 73-105.

[4] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques forrecommender systems," *IEEE Computer*, vol. 42, no. 8, pp. 30-37, Aug. 2009.

[5] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331-370, Nov. 2002.

[6] C. A. Gomez-Uribe and N. Hunt, "The Netflix recommender system: Algorithms, business value, and innovation," *ACM Trans. Management Information Systems*, vol. 6, no. 4, pp. 1-19, Jan. 2016.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic textretrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513-523, 1988.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training ofdeep bidirectional transformers for language understanding," in *Proc. NAACL-HLT 2019*, Minneapolis, MN, 2019, pp. 4171-4186.

[9] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommendersystems: A survey," *IEEE Trans. Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734-749, Jun. 2005.

[10] Kaggle / TMDB, "TMDB 5000 Movie Dataset," 2017. [Online]. Available: https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata.

[11] OMDb, "The Open Movie Database REST API," 2024. [Online]. Available:https://www.omdbapi.com/.

[12] Pallets Projects, "Flask — A Lightweight WSGI Web Application Framework," v3.0.3, 2024. [Online]. Available: https://flask.palletsprojects.com/.

[13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[14] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009. [Online]. Available: https://www.nltk.org/.

[15] Bootstrap Team, "Bootstrap 5 — The World's Most Popular Frontend Framework," v5.3, 2024. [Online]. Available: https://getbootstrap.com/.